

# Admin Panel & SaaS Configuration Guide

## Overview

The admin panel is accessed at `/admin` and provides management of the multi-tenant SaaS platform: distributors, licenses, plans, and tenants. It is separate from the tenant-level ticketing interface.

## 1. Initial Setup

### Seed the Database

```
php artisan migrate
php artisan db:seed
```

This creates:`docker exec -it ticketing-app php artisan route:list | grep track`

| Item             | Details                                                                                 |
|------------------|-----------------------------------------------------------------------------------------|
| Admin user       | <code>admin@example.com</code> / <code>password</code> ( <code>is_admin = true</code> ) |
| Test user        | <code>test@example.com</code> / <code>password</code> (regular tenant user)             |
| 3 Plans          | Start, Business, Enterprise                                                             |
| Demo Distributor | With auto-generated API key                                                             |
| Demo License     | Start plan, 10 seats, 1-year validity                                                   |
| Demo Tenant      | "Demo Company" with test user as owner                                                  |
| 16 Permissions   | Spatie permissions for tenant-level RBAC                                                |

## Access the Admin Panel

1. Navigate to `/admin/login`
  2. Log in with `admin@example.com` / `password`
  3. You'll be redirected to `/admin` (the dashboard)
- 

## 2. Architecture

---

### Authentication

Admins use the same `users` table and `web` guard as regular users. The distinction is the `is_admin` boolean column. The `AdminMiddleware` checks `$user->isAdmin()` and returns 403 if false.

Admin login (`/admin/login`) attempts a standard `Auth::attempt()`, then verifies `isAdmin()` — if the user is not an admin, the session is immediately logged out.

### Entity Hierarchy

```
Distributor
├── License (license_key, seats, status, expires_at, grace_days)
│   └── Plan (features JSON, max_users, max_tickets_per_month)
│       └── Tenant (name, slug, settings, suspended_at)
│           └── Users (pivot: role = owner|admin|member)
```

A tenant gets its plan **through a License**, enabling:

- License key distribution and tracking
- Seat limits independent of the plan
- Expiry dates with configurable grace periods
- Distributor attribution for reseller channels

### Middleware Stack

| Alias              | Class                        | Purpose                              |
|--------------------|------------------------------|--------------------------------------|
| <code>admin</code> | <code>AdminMiddleware</code> | Checks <code>is_admin</code> on user |

---

| Alias   | Class                    | Purpose                                     |
|---------|--------------------------|---------------------------------------------|
| tenant  | EnsureTenantSession      | Resolves tenant from URL slug, sets session |
| feature | CheckPlanFeature         | Gates routes by plan feature flags          |
| portal  | EnsureClientPortalAccess | Validates client portal session             |

### 3. Plans & Features

#### Built-in Plans

| Plan       | Slug       | Max Users | Tickets/Month | Features                    |
|------------|------------|-----------|---------------|-----------------------------|
| Start      | start      | 5         | 100           | None (basic ticketing only) |
| Business   | business   | 25        | 500           | 12 features (see below)     |
| Enterprise | enterprise | unlimited | unlimited     | All 18 features             |

#### Feature Flags

Features are defined in `app/Enums/PlanFeature.php` and stored as a JSON array on the `plans.features` column.

**Business plan features:**

| Feature         | Description                    |
|-----------------|--------------------------------|
| audit_logs      | Ticket Activity History        |
| billing         | Ticket Billing                 |
| spam_management | Mark as Spam                   |
| service_reports | Auto Generated Service Reports |
| attachments     | File Attachments               |
| agent_schedule  | Agent Availability Schedule    |

| Feature                          | Description                 |
|----------------------------------|-----------------------------|
| <code>sla_management</code>      | SLA Management              |
| <code>sla_report</code>          | SLA Compliance Report       |
| <code>email_notifications</code> | Email Notifications         |
| <code>detailed_reporting</code>  | Detailed Reporting & Export |
| <code>knowledge_base</code>      | Knowledge Base              |
| <code>canned_responses</code>    | Canned Responses            |

**Enterprise-only features (in addition to Business):**

| Feature                            | Description                |
|------------------------------------|----------------------------|
| <code>ticket_merging</code>        | Ticket Merging             |
| <code>ticket_reopening</code>      | Ticket Reopening           |
| <code>custom_roles</code>          | Custom Roles & Permissions |
| <code>department_management</code> | Department Management      |
| <code>agent_escalation</code>      | Agent Tiering & Escalation |
| <code>client_comments</code>       | Client-Agent Comments      |

## Managing Plans

Navigate to **Admin > Plans** to create or edit plans.

Each plan has:

- **Name & slug** — slug is used for feature lookups
- **Max users** — leave blank for unlimited
- **Max tickets per month** — leave blank for unlimited
- **Features** — select which feature flags to enable
- **Active** — toggle plan availability

## Feature Checking in Code

```
// In controllers/services
$planService = app(\App\Services\PlanService::class);
$planService->currentTenantHasFeature(\App\Enums\PlanFeature::Billing);

// In routes (middleware)
Route::get('/sla', ...)->middleware('feature:sla_management');

// In Blade templates
@if(app(\App\Services\PlanService::class)-
>currentTenantHasFeature(\App\Enums\PlanFeature::Attachments))
    {{-- Feature UI --}}
@endif
```

Feature checks are cached for 5 minutes per tenant.

---

## 4. Distributors

---

Distributors are reseller entities that issue licenses. Navigate to **Admin > Distributors**.

Each distributor has:

- **Name, email, contact person, phone, address**
- **API key** — auto-generated (dk\_ prefix + 32 random chars)
- **Active status**

Distributors generate licenses via:

```
$distributor->generateLicense($plan, [
    'seats' => 10,
    'expires_at' => now()->addYear(),
    'grace_days' => 7,
]);
```

---

## 5. Licenses

---

Licenses are the bridge between distributors, plans, and tenants. Navigate to **Admin > Licenses**.

## License Lifecycle

pending → active → expired/revoked

| Status  | Description                                                 |
|---------|-------------------------------------------------------------|
| pending | Issued but not yet activated by a tenant                    |
| active  | Activated and in use                                        |
| expired | Past <code>expires_at</code> (may still be in grace period) |
| revoked | Manually revoked by admin                                   |

## License Fields

- **License key** — auto-generated in `XXXX-XXXX-XXXX-XXXX-XXXX` format
- **Distributor** — who issued it
- **Plan** — which plan it grants
- **Seats** — max users for the tenant (can differ from plan's `max_users`)
- **Expires at** — expiration date
- **Grace days** — buffer period after expiry (default: 7 days)

## Validity Check

A license is valid when:

1. Status is `active`
2. Current date is before `expires_at + grace_days`

```
$license->isValid();           // active AND not fully expired
$license->isExpired();          // past expires_at
$license->isInGracePeriod();    // expired but within grace days
$license->isFullyExpired();      // past grace period
$license->daysUntilExpiry();    // days until expires_at
```

## 6. Tenants

Navigate to **Admin > Tenants** to view all tenants.

# Tenant Fields

- **Name** — company/organization name
- **Slug** — URL segment (auto-generated from name), used in `/{{slug}}/dashboard`
- **Logo, primary/accent colors** — branding
- **License** — the active license (determines plan)
- **Settings** — JSON for custom configuration
- **Suspended at** — if set, tenant is suspended

## Tenant Actions (Admin Panel)

| Action      | Description                                           |
|-------------|-------------------------------------------------------|
| View        | See tenant details, usage stats, user list            |
| Suspend     | Sets <code>suspended_at</code> , blocks tenant access |
| Unsuspend   | Clears <code>suspended_at</code>                      |
| Change Plan | Swaps the license's plan                              |
| Impersonate | Log into the tenant as their owner user               |

## Tenant URL Structure

All tenant routes use path-prefix routing:

```
/{slug}/dashboard
/{slug}/tickets
/{slug}/clients
/{slug}/submit-ticket    (public)
/{slug}/track-ticket    (public)
```

The `EnsureTenantSession` middleware resolves the tenant from the `{slug}` URL parameter and sets `session('current_tenant_id')`.

## User Roles Within a Tenant

The `tenant_user` pivot table stores:

| Role   | Capabilities                                |
|--------|---------------------------------------------|
| owner  | Full access, can manage all settings        |
| admin  | Can manage users, settings, and all tickets |
| member | Standard agent, can manage assigned tickets |

Spatie Permission's team mode is used, where `setPermissionsTeamId($tenant->id)` scopes all role/permission checks to the specific tenant.

## Tenant Data Isolation

All tenant-owned models (Ticket, Client, Department, etc.) use a `TenantScope` global scope that filters by `session('current_tenant_id')`. This ensures complete data isolation between tenants.

## 7. Impersonation

Admins can impersonate any tenant to debug issues:

1. Go to **Admin > Tenants > [Tenant] > Impersonate**
2. Admin is logged into the tenant's context as the first owner
3. A banner appears with a "Stop Impersonation" button
4. Session keys set: `admin_impersonating`, `admin_return_url`, `current_tenant_id`
5. Clicking "Stop" restores the admin session and redirects back

## 8. Admin Dashboard

The admin dashboard (`/admin`) shows:

- **Total tenants, active licenses, revenue metrics**
- **Expiring licenses** (within 30 days) — with alerts
- **Plan distribution chart** — how many tenants on each plan
- **Top tenants** by user count
- **Recent tenants** — newest registrations

---

## 9. Adding a New Tenant (Manual Flow)

---

1. **Create a Distributor** (or use an existing one)
2. **Create a License** under that distributor:
  - Select a plan
  - Set seats and expiry
3. **Register a new user** (or use existing) through the app's registration
4. During registration/onboarding, the user provides a license key
5. The license is activated, creating the tenant and associating the user as **owner**

Or via the seeder flow for development:

```
$distributor = Distributor::create(['name' => 'My Distributor']);
$license = $distributor->generateLicense($plan, ['seats' => 10]);
$tenant = Tenant::create(['name' => 'New Company']);
$license->activate($tenant);
$tenant->addUser($user, 'owner');
```

---

## 10. Environment Configuration

---

Key **.env** variables relevant to SaaS:

```
APP_URL=http://localhost:8080      # Base URL (tenant slugs are appended as path)
DB_CONNECTION=mysql               # Shared database (single-DB multi-tenancy)
CACHE_STORE=database              # or redis – used for feature flag caching
SESSION_DRIVER=database           # or redis
```

No subdomain configuration is needed — all tenant routing uses path prefixes.

---

## 11. Code Structure

---

### Directory Layout

```

app/
├── Console/Commands/
│   └── SendSlaBreachWarnings.php           # Scheduled artisan command
├── Enums/
│   └── PlanFeature.php                     # Backed string enum – 18 feature flags
├── Http/
│   ├── Controllers/
│   │   ├── Admin/                         # Admin panel controllers
│   │   │   ├── AuthController.php
│   │   │   ├── DashboardController.php
│   │   │   ├── DistributorController.php
│   │   │   ├── LicenseController.php
│   │   │   ├── PlanController.php
│   │   │   └── TenantController.php
│   │   ├── Auth/                         # Breeze auth controllers
│   │   └── [Tenant-level controllers]     # TicketController, ClientController,
etc.
├── Middleware/
│   ├── AdminMiddleware.php                # Checks user->isAdmin()
│   ├── CheckPlanFeature.php               # Gates routes by PlanFeature
│   ├── EnsureClientPortalAccess.php       # Portal auth + tenant resolution
│   ├── EnsureTenantSession.php            # URL slug → session tenant resolution
│   └── SetTenantUrlDefaults.php            # Injects slug into URL::defaults()
├── Requests/                             # Form Request validation classes
├── Models/
│   ├── Scopes/
│   │   └── TenantScope.php                # Global scope for data isolation
│   ├── Traits/
│   │   ├── BelongsToTenant.php            # Auto-scopes models to current tenant
│   │   └── HasTenants.php                 # Multi-tenant user relationships
│   └── [All Eloquent models]
├── Notifications/                        # Queued notification classes
├── Services/                             # Business logic services
└── View/Components/                      # Layout components

```

## Route Files

| File                           | Purpose                                                                         |
|--------------------------------|---------------------------------------------------------------------------------|
| <code>routes/web.php</code>    | Admin panel routes, auth routes, home/registration                              |
| <code>routes/tenant.php</code> | All/ <code>{slug}</code> /... tenant routes (dashboard, tickets, clients, etc.) |
| <code>routes/portal.php</code> | Client portal routes under/ <code>{tenant}</code> /portal/...                   |

## 12. Packages Used

# Production Dependencies

| Package                    | Version | Purpose                                                         |
|----------------------------|---------|-----------------------------------------------------------------|
| laravel/framework          | v12     | Core framework                                                  |
| spatie/laravel-permission  | v6.24   | Role/permission system with <b>teams mode</b> (per-tenant RBAC) |
| barryvdh/laravel-dompdf    | v3.1    | PDF generation for service reports                              |
| aws/aws-sdk-php            | v3      | AWS SDK for S3 file storage                                     |
| league/flysystem-aws-s3-v3 | v3      | Laravel S3 filesystem driver                                    |
| laravel/tinker             | latest  | REPL debugging                                                  |

# Dev Dependencies

| Package         | Version | Purpose                              |
|-----------------|---------|--------------------------------------|
| laravel/breeze  | latest  | Auth scaffolding (Blade starter kit) |
| laravel/sail    | v1      | Docker dev environment               |
| laravel/pint    | v1      | Code style formatter                 |
| laravel/boost   | v2      | MCP/dev tooling                      |
| phpunit/phpunit | v11     | Testing framework                    |
| fakerphp/faker  | latest  | Test data generation                 |

# No Dedicated Tenancy Package

Multi-tenancy is **entirely custom-built** — no `stancl/tenancy` or `spatie/laravel-multitenancy`. It uses:

- `BelongsToTenant` trait + `TenantScope` global scope for data isolation
- Session-based tenant switching (`session('current_tenant_id')`)
- Path-prefix routing (`{slug}/...`) resolved by `EnsureTenantSession` middleware

---

## 13. Custom Traits

---

### BelongsToTenant — app/Models/Traits/BelongsToTenant.php

Used on all tenant-owned models. Provides:

- **bootBelongsToTenant()** — Registers `TenantScope` as a global scope on every query
- **creating event** — Auto-fills `tenant_id` from `session('current_tenant_id')` if not set
- **tenant(): BelongsTo** — Relationship to `Tenant`
- **scopeForTenant(\$query, int \$tenantId)** — Query a specific tenant
- **scopeWithoutTenantScope(\$query)** — Remove tenant filter

### HasTenants — app/Models/Traits/HasTenants.php

Used only on the `User` model. Provides:

- **tenants(): BelongsToMany** — Pivot with `role` and `joined_at` columns
- **currentTenant() / setCurrentTenant() / clearCurrentTenant()** — Session-based tenant switching
- **belongsToTenant(Tenant) / roleInTenant(Tenant)** — Membership checks
- **isOwnerOf(Tenant) / isAdminOf(Tenant)** — Role checks
- **ownedTenants() / adminTenants()** — Filtered pivot queries
- **ensureCurrentTenant()** — Auto-sets tenant from session or first available

---

## 14. Trait Usage Per Model

---

| Model | Traits                                                         |
|-------|----------------------------------------------------------------|
| User  | HasFactory, HasRoles (Spatie), HasTenants (custom), Notifiable |

---

| Model                   | Traits                                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------|
| <b>Ticket</b>           | <code>HasFactory</code> , <code>SoftDeletes</code> , <code>BelongsToTenant</code> (custom) |
| <b>Client</b>           | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>Department</b>       | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>TicketCategory</b>   | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>TicketComment</b>    | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>TicketTask</b>       | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>Product</b>          | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>SlaPolicy</b>        | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>CannedResponse</b>   | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>KbArticle</b>        | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>KbCategory</b>       | <code>HasFactory</code> , <code>BelongsToTenant</code> (custom)                            |
| <b>ServiceReport</b>    | <code>BelongsToTenant</code> (custom)                                                      |
| <b>AgentSchedule</b>    | <code>BelongsToTenant</code> (custom)                                                      |
| <b>AppSetting</b>       | <code>BelongsToTenant</code> (custom)                                                      |
| <b>Tenant</b>           | <code>HasFactory</code>                                                                    |
| <b>Plan</b>             | <code>HasFactory</code>                                                                    |
| <b>License</b>          | <code>HasFactory</code>                                                                    |
| <b>Distributor</b>      | <code>HasFactory</code>                                                                    |
| <b>TicketAssignment</b> | <i>(none — bare Model)</i>                                                                 |
| <b>TicketEscalation</b> | <i>(none — bare Model)</i>                                                                 |
| <b>TicketHistory</b>    | <i>(none — bare Model)</i>                                                                 |

### Key observations:

- Only `Ticket` uses `SoftDeletes` — all other models use hard deletes
- Only `User` uses `Notifiable` — client notifications use `Notification::route('mail', $email)`
- All queued notifications implement `ShouldQueue` with the `Queueable` trait

# 15. Service Classes

All in `app/Services/`:

| Service                     | Responsibilities                                                                                                                                          |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TicketService</b>        | Core ticket lifecycle: create, update, assign, change status, close, billing, spam, history. Wires <code>SlaService</code> and <code>PlanService</code> . |
| <b>SlaService</b>           | Assign SLA policies to tickets, find overdue tickets, breach warning candidates, compliance reporting.                                                    |
| <b>EscalationService</b>    | Escalate/de-escalate tickets between tiers (tier_1 → tier_2 → tier_3). Records <code>TicketEscalation</code> + history.                                   |
| <b>TicketMergeService</b>   | Merge/unmerge tickets in a DB transaction, moving tasks and comments to the target ticket.                                                                |
| <b>PlanService</b>          | Check if a tenant has a <code>PlanFeature</code> , read features from cache (5-min TTL), clear cache, check current session tenant.                       |
| <b>TenantRoleService</b>    | Bootstrap default roles/permissions for a tenant using Spatie's teams mode. Defines <code>PERMISSIONS</code> and <code>ROLE_PERMISSIONS</code> constants. |
| <b>ReportService</b>        | Ticket volume, resolution time, department, and agent performance reports. CSV export via streamed response.                                              |
| <b>ServiceReportService</b> | Generate PDF service reports via <code>barryvdh/laravel-dompdf</code> , store to S3/disk.                                                                 |
| <b>TenantUrlHelper</b>      | Generate absolute URLs for a tenant's slug-prefixed paths.                                                                                                |

# 16. Global Scope — TenantScope

File: `app/Models/Scopes/TenantScope.php`

```
class TenantScope implements Scope
{
    public function apply(Builder $builder, Model $model): void
```

```

{
    $tenantId = static::getCurrentTenantId();
    if ($tenantId !== null) {
        $builder->where($model->getTable() . '.tenant_id', $tenantId);
    }
}

public static function getCurrentTenantId(): ?int
{
    return session('current_tenant_id');
}
}

```

Applied automatically on every model using `BelongsToTenant` (14 models). This ensures **complete data isolation** — a query on `Ticket::all()` only returns tickets for the current tenant.

### Bypass when needed:

```

// Remove scope for a single query
Ticket::withoutGlobalScope(TenantScope::class)->get();

// Query a specific tenant
Ticket::query()->forTenant($tenantId)->get();

```

---

## 17. Spatie Permission Configuration

---

**File:** `config/permission.php`

Key settings:

- **Teams mode enabled:** `'teams' => true`
- **Team foreign key:** `'team_foreign_key' => 'tenant_id'`

This means all roles and permissions are scoped per tenant. When checking `$user->hasRole('admin')`, it only checks within the current tenant context set by `setPermissionsTeamId($tenant->id)`.

## 16 Permissions (from `TenantRoleService`)

view\_tickets, create\_tickets, edit\_tickets, delete\_tickets,  
assign\_tickets, manage\_clients, manage\_categories,  
manage\_departments, manage\_products, manage\_sla,  
view\_reports, manage\_settings, manage\_roles,  
manage\_schedules, manage\_billing, manage\_knowledge\_base

# Default Roles

| Role   | Permissions                                      |
|--------|--------------------------------------------------|
| admin  | All 16 permissions                               |
| agent  | view/create/edit tickets, assign, manage clients |
| viewer | view_tickets only                                |